

Technical Documentation

VirusTotal Android Malware Risk Profiling

Final project using Python, MongoDB, and VirusTotal Android APK reports

Student	Shayan
Project type	Data engineering and exploratory data analysis
Main technologies	Python, pandas, matplotlib, MongoDB

1. Project Overview

In this project I worked with the VirusTotal Android APK dataset and built a small end-to-end pipeline. The starting point was a ZIP file containing nested JSON reports. My goal was to convert those reports into clean tables, analyze the antivirus consensus, and also prepare a MongoDB structure for querying the same data in a NoSQL way.

The main question I used for the project was: how can I transform semi-structured VirusTotal Android reports into an analysis-ready dataset, measure the level of agreement between antivirus engines, and identify the strongest malware or risk patterns in the samples?

2. What I Built

The project is divided into three practical parts:

- A Python data engineering script that reads the original VirusTotal reports and normalizes the data.
- A second Python analysis script that adds engineered features, correlations, and extra figures.
- A MongoDB import and query layer, so the project also covers document storage and NoSQL analysis.

2.1 Project folder structure

Path	Purpose in the project
src/analyze_virustotal.py	Main pipeline. Reads the ZIP file, parses JSON reports, creates CSV files, computes metrics, and generates charts.
src/advanced_python_analysis.py	Adds feature engineering, consensus bands, correlation analysis, and additional charts.
src/import_to_mongodb.py	Loads the normalized CSV outputs into MongoDB and creates useful indexes.
mongodb/queries.js	Contains example MongoDB queries for top samples, risk categories, engines, tags, and in-the-wild URLs.
outputs/	Generated CSV files, JSON metrics, Markdown summaries, and chart images.
docs/	Written report and presentation outline generated for the submission.

3. Dataset Description

The input dataset is VTAndroid.zip. It contains 157 JSON reports for Android APK files. Each JSON file contains several kinds of information: file hashes, file size, dates, tags, submission metadata, community information, and a nested scans object with the result from each antivirus engine.

Dataset item	Value after processing
APK reports analyzed	157
Normalized engine-sample rows	9564
Distinct antivirus engines	66
Detection positives range	15 to 33 engines
Median detection ratio	0.323
Samples with in-the-wild URLs	44
Maximum times submitted	16999

4. Methodology

4.1 Data engineering pipeline

I kept the pipeline simple and reproducible. The main script opens the ZIP file directly, reads every JSON report, and then builds two main tables. The first table is sample-level, with one row per APK. The second table is scan-level, with one row for each antivirus engine result for each APK.

1. Open VTAndroid.zip and find all JSON reports inside it.
2. Parse each report and extract hashes, file metadata, tags, submission fields, and scan results.
3. Compute the detection ratio from positives and total engines.
4. Assign an operational risk category using the detection ratio.
5. Normalize the nested antivirus scan dictionary into a long table.
6. Export CSV files, a JSON metrics file, Markdown summaries, and PNG charts.

4.2 Detection ratio and risk categories

The main score I used is the detection ratio. I used it because it is easy to understand and it is directly available from the VirusTotal report fields:

```
detection_ratio = positives / total_engines
```

I did not use this as a perfect malware label. I used it as an operational consensus score, because the dataset does not include an independent ground-truth label.

Risk category	Rule used in the project
High-confidence malicious	detection_ratio >= 0.50
Suspicious	0.10 <= detection_ratio < 0.50
Low consensus	detection_ratio < 0.10

5. MongoDB Design

For the MongoDB part, I used two collections. I chose this because the project needs both sample-level questions and antivirus-engine-level questions. Keeping them separate made the queries easier and avoided putting too much repeated scan information inside each sample document.

Collection	Granularity	Main fields
vt_samples	One document per APK sample	sha256, positives, total_engines, detection_ratio, risk_category, tags, size_bytes, times_submitted, itw_url_count
vt_engine_scans	One document per engine result	sha256, engine, detected, result, engine_version, engine_update, scan_date

The importer creates indexes on fields that are useful for the queries, for example sha256, detection_ratio, risk_category, tags, engine, detected, and scan_date.

Example query used to find the samples with the strongest antivirus consensus:

```
db.vt_samples.find({}, { _id: 0, sha256: 1, detection_ratio: 1, risk_category: 1 })
  .sort({ detection_ratio: -1 })
  .limit(10);
```

6. Main Results

6.1 Antivirus consensus

The dataset is strongly concentrated in the suspicious range. The minimum number of positive detections is 15, the median is 19, and the maximum is 33. The median detection ratio is 0.323.

Based on the thresholds I used, 156 samples are suspicious and 1 sample is high-confidence malicious. There were no low-consensus samples in this dataset.

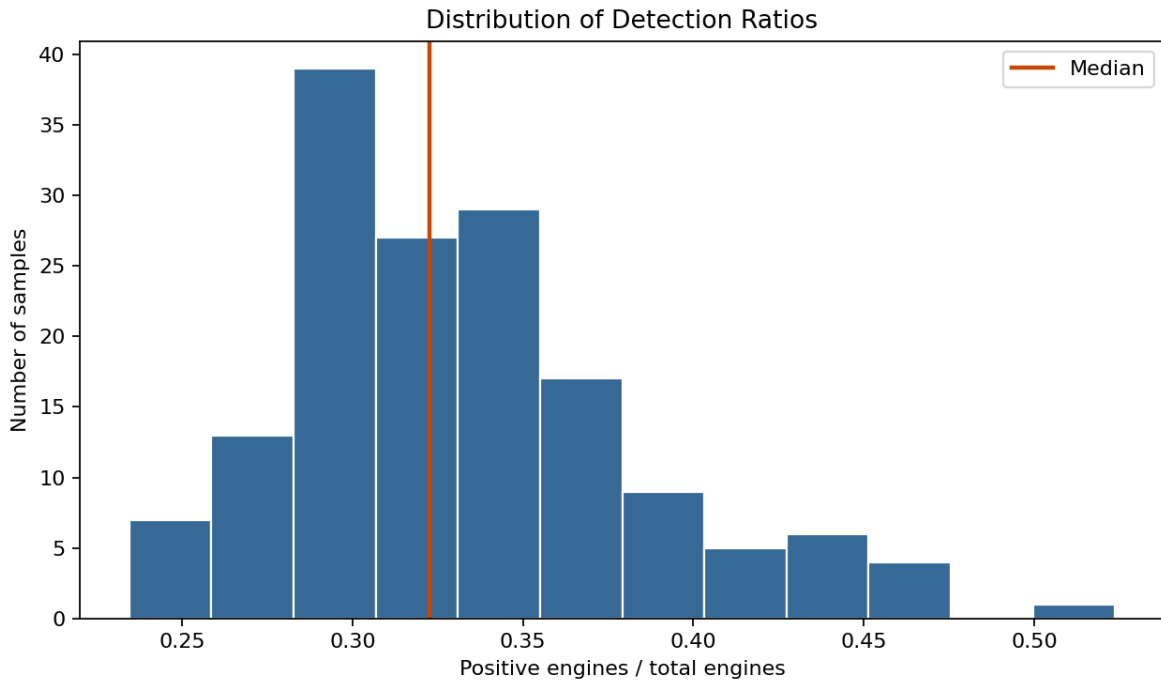


Figure 1. Distribution of detection ratios. The median is in the suspicious range.

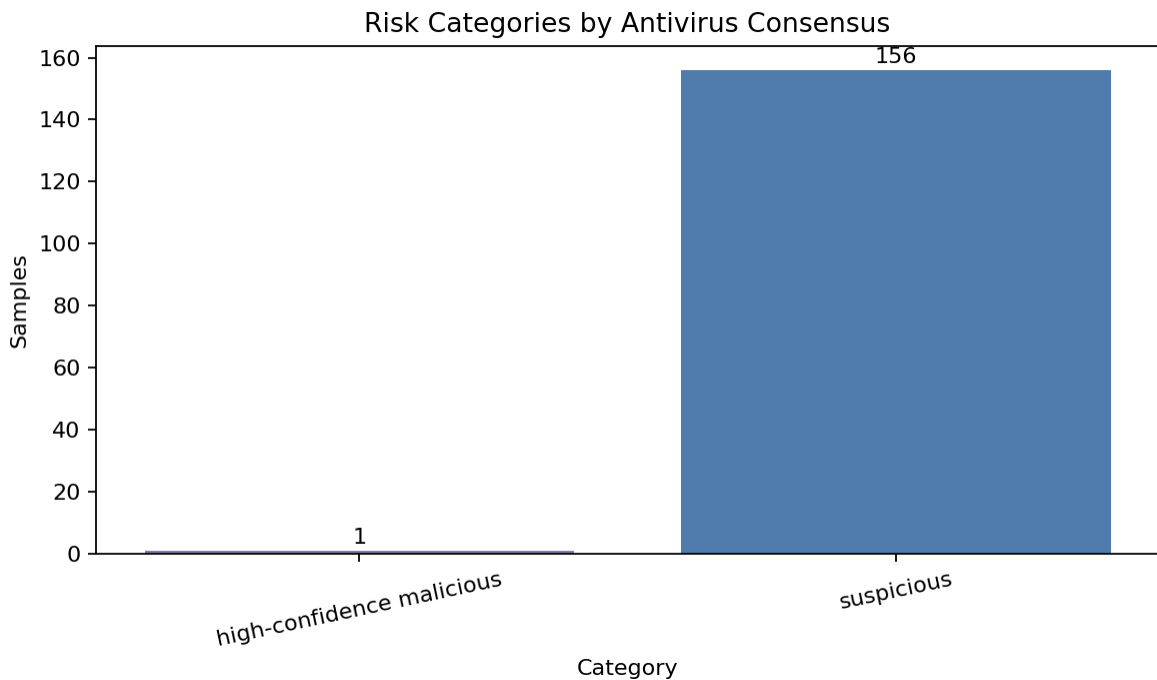


Figure 2. Risk category distribution using my detection-ratio thresholds.

6.2 Engines with more positive detections

The table below shows the engines that produced the highest number of positive detections. I also included the number of samples seen because not every engine appears in every report.

Engine	Detections	Samples seen	Detection rate
K7GW	151	157	0.962
McAfee	146	150	0.973
Avira	142	157	0.904
DrWeb	142	157	0.904
Fortinet	142	157	0.904
Ikarus	140	143	0.979
Kaspersky	140	151	0.927
Cynet	140	155	0.903
CAT-QuickHeal	139	157	0.885
McAfee-GW-Edition	138	148	0.932

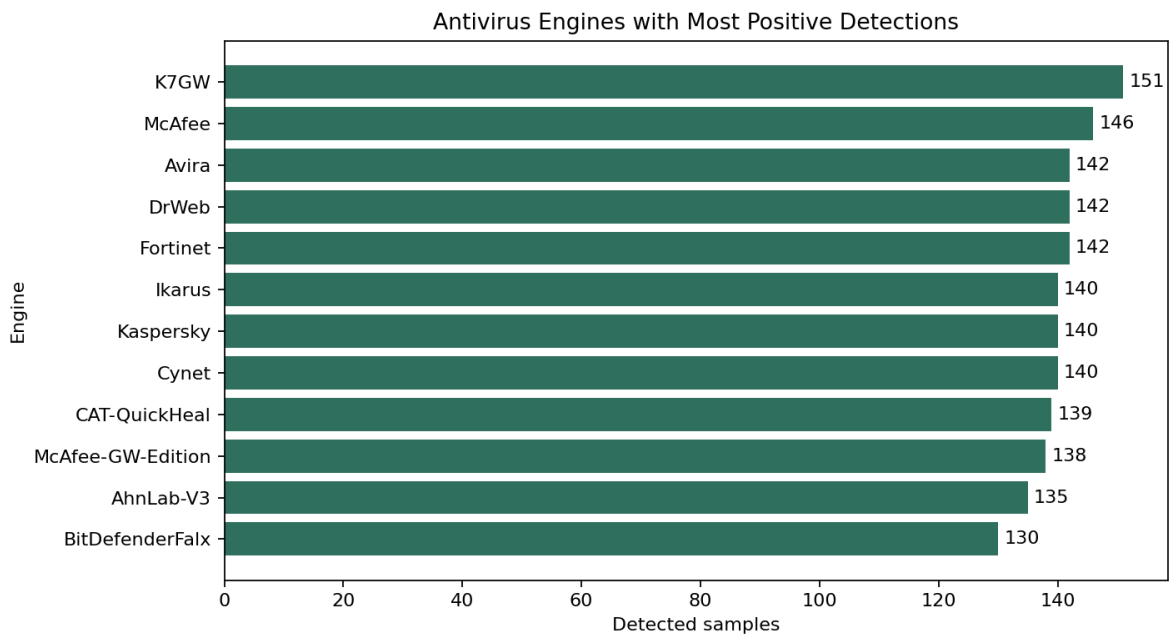


Figure 3. Antivirus engines with the highest number of positive detections.

6.3 Tags and detection-name patterns

The tags give a quick technical description of the samples. Some are general, like android and apk, while others are more useful for risk analysis, like reflection, telephony, obfuscated, contains-elf, and runtime-modules.

Tag	Count
android	157
apk	145
reflection	42
via-tor	31
telephony	15
dex	12
contains-elf	10
runtime-modules	8
obfuscated	7
mysql-communication	5

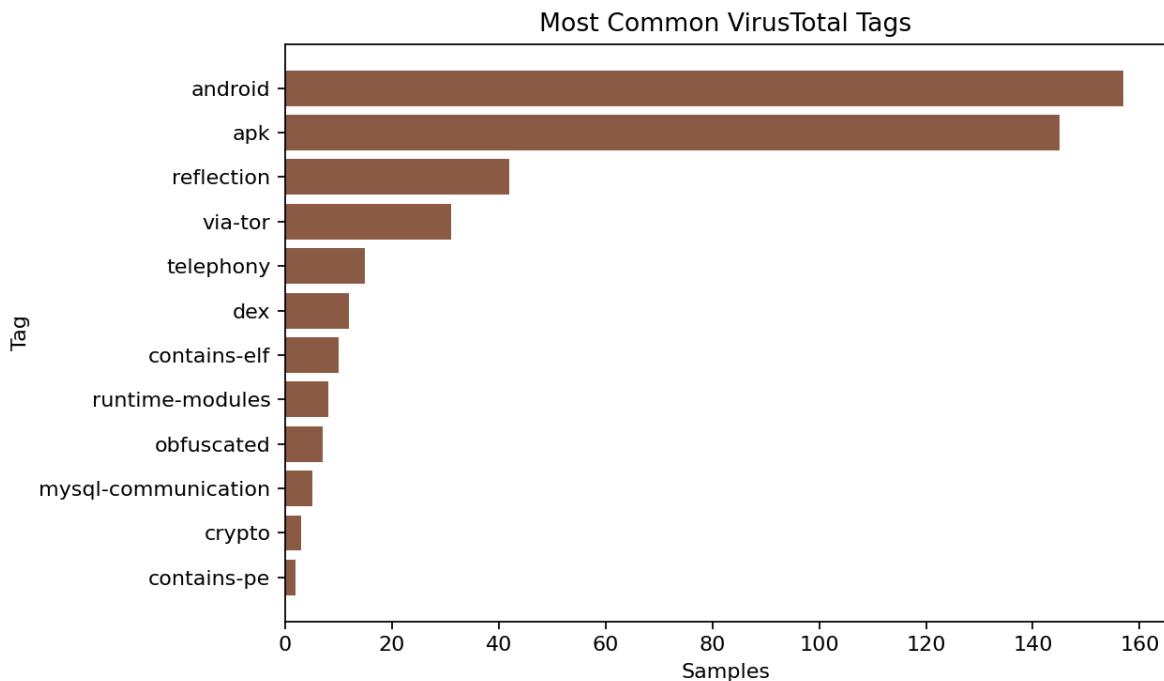


Figure 4. Most common VirusTotal tags in the analyzed samples.

I also extracted frequent terms from antivirus detection names. This is not perfect because every vendor uses its own naming style, but it still gives a useful view of repeated patterns.

Detection term	Count
adware	601
artemis	281
riskware	219
mobby	202
notifier	196

The most repeated terms point mainly to adware, riskware, hidden advertising, SMS registration behavior, and potentially unwanted applications. So in my interpretation, the dataset is more useful for risk profiling than for a clean benign-versus-malware classifier.

7. Extended Python Analysis

After the first normalization step, I added a second Python script to create extra features. This helped me check whether simple metadata fields have a visible relation with the antivirus consensus score.

- `size_mb`: file size converted from bytes to MB.
- `has_itw_urls`: whether VirusTotal includes in-the-wild URL evidence.
- `is_repeatedly_submitted`: whether the sample has more submissions than the dataset median.
- `tag_count`: number of tags attached to the sample.
- `submission_name_count`: number of names observed in submissions.
- `consensus_band`: binned version of the detection ratio for easier group analysis.

7.1 Consensus band summary

Band	Samples	Avg. ratio	Avg. size MB	Avg. submissions	ITW URL samples	Avg. tag count
low-medium	4	0.239	8.508	16.750	4	5.500
medium	107	0.308	5.189	164.794	21	2.701
medium-high	45	0.392	4.025	8.778	19	2.800
high	1	0.524	2.278	1.000	0	4.000

One clear point here is that most samples are in the medium band. The one high-band sample is not enough to generalize, but it is still useful as an example of stronger engine consensus.



Figure 5. File size compared with detection ratio. Points with in-the-wild URL evidence are highlighted in the generated figure.

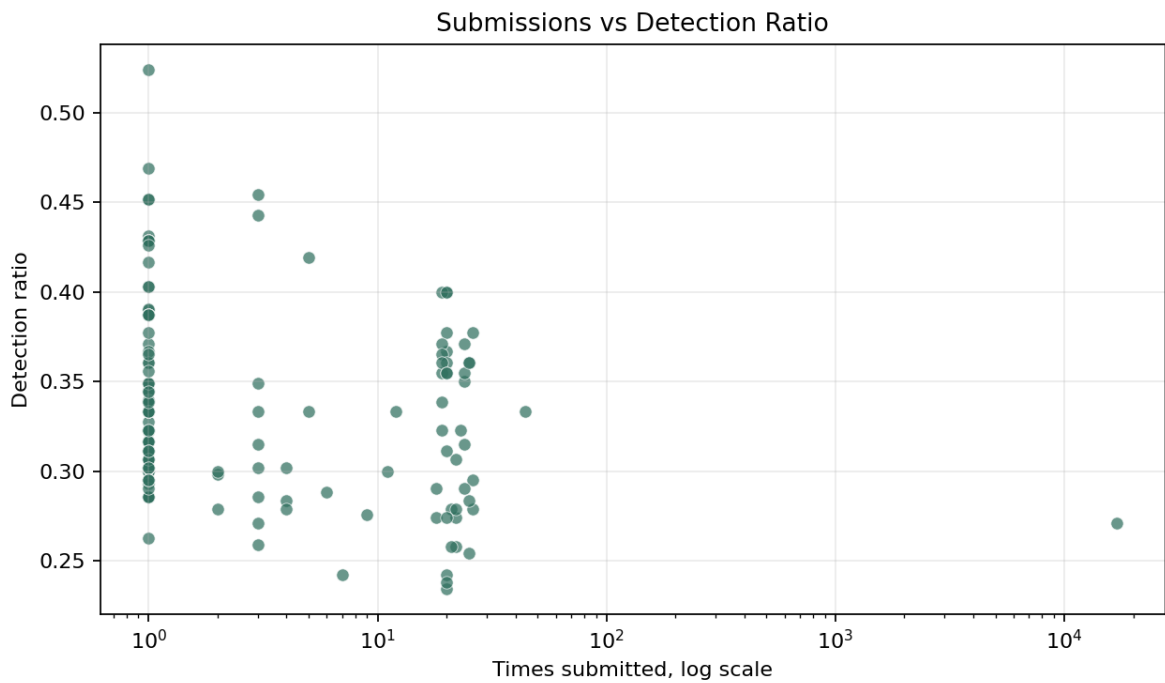


Figure 6. Number of submissions compared with detection ratio using a logarithmic x-axis.

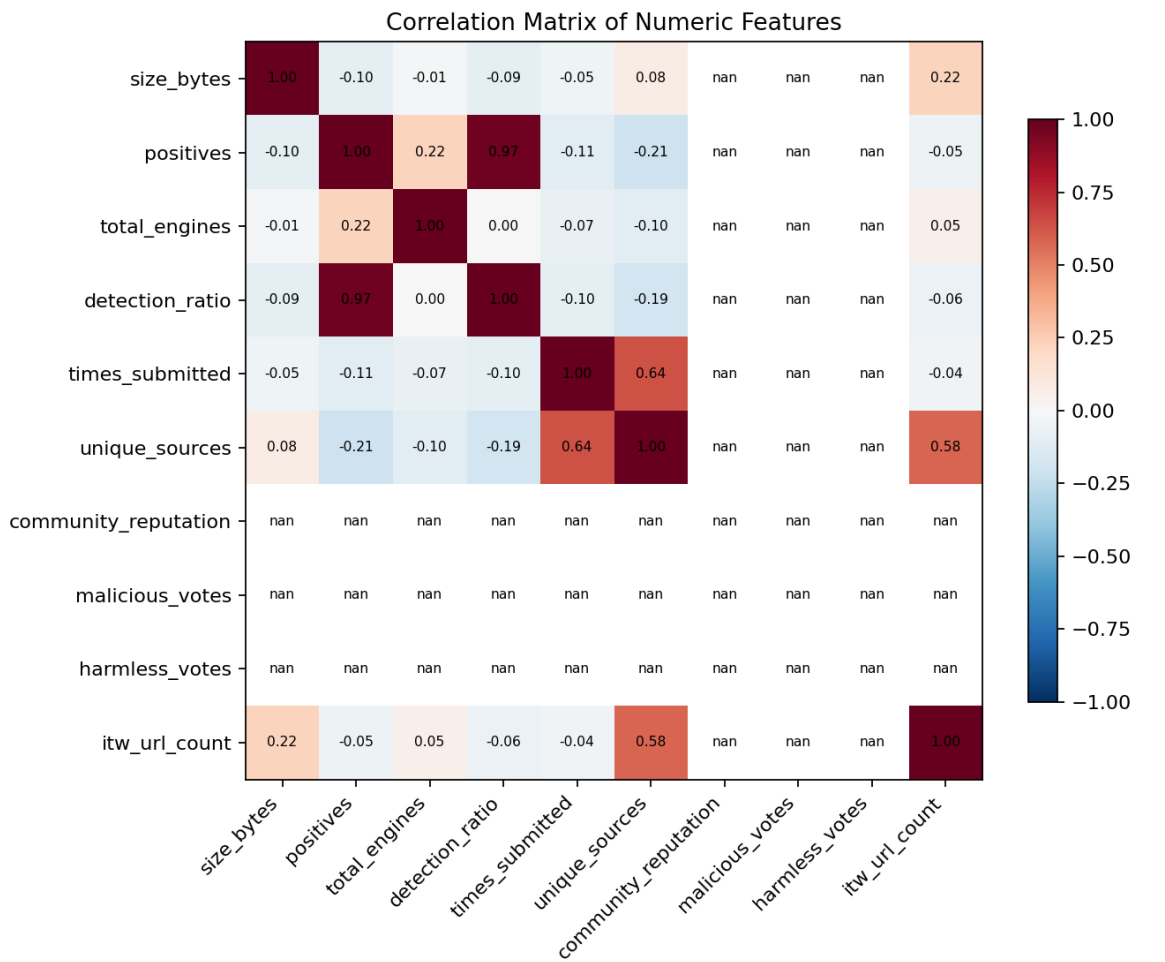


Figure 7. Exploratory correlation matrix for numeric fields.

The strongest numerical relation with detection_ratio is positives, which is expected because positives is part of the detection-ratio formula. For the other metadata fields, I treated the correlations only as exploratory signals, not as proof of causation.

8. Generated Outputs

The project generates these main outputs:

Output file	Description
samples_summary.csv	Sample-level table with one row per APK.
engine_scans_long.csv	Long table with one row per antivirus engine result.
top_detection_engines.csv	Ranked engine summary.
top_tags.csv	Most common VirusTotal tags.
top_detection_terms.csv	Most common terms extracted from detection names.
project_metrics.json	Main numeric metrics used in the report.
python_feature_table.csv	Feature-engineered sample table.
consensus_band_summary.csv	Summary by detection-ratio band.
numeric_feature_correlations.csv	Correlation matrix exported as CSV.
outputs/charts/*.png	Charts used for the report and presentation.

9. How to Run the Project

From the project folder, I used these commands:

```
python -m pip install -r requirements.txt
python .\src\analyze_virustotal.py
python .\src\advanced_python_analysis.py
```

The first script expects the original dataset in this relative path:

```
..\Python for Data Science\VTAndroid.zip
```

For the optional MongoDB import, I used this command after starting MongoDB locally:

```
python .\src\import_to_mongodb.py --uri "mongodb://localhost:27017"
```

10. Validation Checks I Used

I used simple validation checks to make sure the generated data made sense. The most important checks were:

- The number of sample rows is 157, matching the number of JSON reports processed.
- The long scan table has 9564 rows, which shows that the nested antivirus scan objects were expanded correctly.
- There are 66 distinct antivirus engines in the normalized scan table.
- The detection ratio is calculated only from positives and total_engines, so it can be checked manually for any row.
- The MongoDB collections are loaded from the same CSV outputs, so Python and MongoDB use the same normalized source.

11. Conclusion

This project helped me practice the full path from semi-structured JSON data to analysis-ready outputs. I normalized nested VirusTotal reports, generated sample-level and engine-level tables, computed a consensus-based risk score, created visualizations, and prepared MongoDB collections and queries.

The main result is that the dataset contains mostly suspicious Android APKs with moderate antivirus agreement. Only one sample reached my high-confidence malicious threshold, while the rest remained in the suspicious range. The repeated detection terms and tags suggest that many samples are related to adware, riskware, hidden advertising, telephony behavior, and SMS registration patterns.

My conclusion is that this dataset is useful for risk profiling and exploratory analysis, but it should not be treated as a clean supervised malware classification dataset unless extra benign samples and ground-truth labels are added.

Appendix A. Short Code Notes

Important logic from the project, simplified for explanation:

```
def risk_category(detection_ratio):  
    if detection_ratio >= 0.50:  
        return "high-confidence malicious"  
    if detection_ratio >= 0.10:  
        return "suspicious"  
    return "low consensus"
```

The scan-level table is created by looping over the nested scans dictionary in each VirusTotal report:

```
for engine, scan in report.get("scans", {}).items():  
    scan_rows.append({  
        "sha256": report.get("sha256"),  
        "engine": engine,  
        "detected": bool(scan.get("detected")),  
        "result": scan.get("result")  
    })
```